Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3 | Plan de Pruebas y Reglas para el Versionado e Incidencias.

Aseguramiento de Calidad.

Ingeniería en Desarrollo de Software

academiaglobal

TUTOR:

ALUMNO: Manuel Marin Sanchez.

FECHA: 06/01/2026.

Indicé.

Portada ……. 1
Indicé …… 2
Introducción ……. 3
Descripción ……. 4
Justificación ……. 5
Desarrollo
Plan de pruebas integral ……... 6
Reglas para el versionado e incidencias …… 7
Conclusión …….. 8
Referencias ……. 9

Introducción.

Este documento presenta el plan integral de pruebas y las reglas de versionado y manejo de incidencias para el sistema de control de inventario de la compañía Cajas y Derivados S.A. de C.V., desarrollado en las actividades previas del curso de Aseguramiento de la Calidad. El objetivo principal es garantizar la calidad del software mediante un enfoque estructurado que incluye cronograma detallado, definición clara de roles, estándares de calidad y procesos sistemáticos de control de cambios. La implementación de un plan de pruebas sistemático y un sistema de versionado robusto permite reducir riesgos, asegurar la confiabilidad del producto, facilitar el mantenimiento futuro y cumplir con las expectativas tanto de los usuarios finales como de la dirección de la empresa. Este trabajo se fundamenta en las mejores prácticas de aseguramiento de la calidad (SQA) y gestión de configuración, alineadas con estándares reconocidos como IEEE 829 para documentación de pruebas e ITIL para gestión de servicios e incidencias. Además, se integran conceptos clave abordados a lo largo del curso, como atributos de calidad, técnicas de prueba (caja negra, caja blanca, pruebas OO y web), diseño de casos de prueba y gestión de cambios. La elaboración de este proyecto no solo consolida los conocimientos teóricos, sino que también simula un escenario real de desarrollo de software, preparando al estudiante para enfrentar desafíos profesionales en entornos donde la calidad, la trazabilidad y la mejora continua son pilares fundamentales del éxito del producto.

Descripción.

El proyecto consiste en un sistema web multiplataforma para el control integral de inventario, clientes y empleados de la empresa Cajas y Derivados S.A. de C.V., desarrollado con tecnologías modernas como HTML5, CSS, Python bajo el patrón MVC y PostgreSQL como gestor de base de datos. En actividades anteriores se identificaron atributos de calidad esenciales (funcionalidad, usabilidad, confiabilidad, seguridad, eficiencia, portabilidad y mantenibilidad), se diseñaron casos de prueba específicos (para autenticación, altas de empleados y productos, generación de reportes) y se detectaron defectos críticos como la falta de validación de duplicados y la generación de reportes corruptos. A partir de ese diagnóstico, en esta etapa final se diseña un plan de pruebas integral que organiza de manera secuencial y temporal las actividades de validación, asigna roles específicos (Líder de Calidad, Testers, Desarrolladores), establece un cronograma detallado de seis semanas y define estándares de calidad medibles. Adicionalmente, se proponen reglas de versionado basadas en *Semantic Versioning* (MAJOR.MINOR.PATCH-FASE) para gestionar las liberaciones del software, y un procedimiento estructurado de manejo de incidencias que incluye reporte, clasificación, solución y documentación de cada error o cambio. Este enfoque integral busca asegurar que el software no solo cumpla con los requerimientos funcionales y técnicos, sino que también ofrezca una experiencia de usuario fluida, segura y confiable, facilitando además la escalabilidad y el mantenimiento futuro del sistema.

Justificación.

La implementación de un plan de pruebas integral y de reglas claras para el versionado y manejo de incidencias es fundamental para el éxito del proyecto y la calidad del software por múltiples razones estratégicas, técnicas y operativas. Primero, un plan de pruebas estructurado permite detectar y corregir defectos en etapas tempranas del desarrollo, lo que reduce significativamente los costos y tiempos asociados con las correcciones tardías, tal como señala Pressman (2005) en su modelo de costos de calidad. Segundo, el control de cambios mediante un versionado semántico y una gestión disciplinada de incidencias evita confusiones en las versiones del software, permite rastrear cada modificación y asegura que todos los integrantes del equipo trabajen sobre la misma base de código. Tercero, la definición clara de roles, responsabilidades y un cronograma realista mejora la coordinación y colaboración entre desarrolladores, testers y administradores, optimizando el uso de recursos humanos y técnicos. Cuarto, el seguimiento de estándares reconocidos como IEEE 829 para planes de prueba e ITIL para gestión de incidencias no solo garantiza profesionalismo y repetibilidad, sino que también facilita la auditoría y certificación del sistema. Quinto, un sistema bien documentado, versionado y con un historial claro de incidencias resueltas facilita enormemente el mantenimiento futuro, la escalabilidad y la incorporación de nuevas funcionalidades. En conjunto, estas prácticas transforman el desarrollo de software en un proceso predecible, controlado y orientado a la excelencia, alineando los objetivos técnicos con las necesidades del negocio.

Desarrollo.
Plan de pruebas integral.

Objetivo: Verificar que el sistema cumpla con los requerimientos funcionales, técnicos y de calidad definidos.
Roles del equipo:
Líder de calidad: Coordina el plan de pruebas, revisa resultados.
Tester senior: Diseña casos de prueba, ejecuta pruebas complejas.
Tester junior: Ejecuta pruebas de regresión y documenta incidencias.
Desarrollador: Corrige defectos reportados y valida soluciones.
Administrador de BD: Asegura integridad de datos durante pruebas.

Estándares de calidad:

Cumplimiento al 100% de requerimientos funcionales.
Tiempo de respuesta menor a 3 segundos en transacciones críticas.
Cobertura de pruebas mayor a 90% en módulos clave.
Cero defectos críticos en liberación a producción.

Cronograma de actividades (semanas 1 a 6).

Fases	Actividades	Sem 1	Sem 2	Sem 3	Sem 4	Sem 5	Sem 6
Establecer Objetivos.	Analizar requerimientos. Definir criterios de aceptación.	* *					
Planificación.	Asignación de roles. Diseño de casos de prueba. Elaboración de cronograma.		* * *				
Ejecución.	Pruebas de unidad. Pruebas de integración. Pruebas de sistema. Pruebas de aceptación (beta).			* *	* *	* *	
Seguimiento y Control.	Revisión de resultados. Gestión de incidencias.			*	*	* *	
Cierre.	Entrega del sistema. Evaluación final y retroalimentación.						* *

Reglas para el versionado e incidencias.

Reglas de versionado (Semantic Versioning):

Formato: MAJOR.MINOR. PATCH-FASE
Ejemplo: 1.2.3-beta

- **MAJOR**: Cambios incompatibles con versiones anteriores.
- **MINOR**: Nuevas funcionalidades compatibles.
- **PATCH**: Corrección de errores menores.
- **FASE**: alpha (desarrollo interno), beta (pruebas), rc (release candidate), stable (producción).

Flujo de versionado:

1. Desarrollo → 0.1.0-alpha
2. Pruebas iniciales → 0.2.0-beta
3. Corrección de errores → 0.2.1-beta
4. Liberación estable → 1.0.0-stable

Manejo de incidencias:

1. Reporte: Todo defecto se registra en un sistema de seguimiento (ej.: Jira) con ID único.
2. Clasificación:
   - Crítica: Bloqueante, se atiende en < 24 h.
   - Alta: Funcionalidad afectada, se atiende en < 72 h.
   - Media: Problema de usabilidad, se agenda.
   - Baja: Mejora o ajuste menor.
3. Solución: Asignada a desarrollador, se prueba y cierra con versión de corrección.
4. Documentación: Cada incidencia debe incluir pasos para reproducir, entorno y evidencia.

Ejemplo de registro de incidencia:

- ID: INC-2024-001
- Descripción: No se valida duplicado en alta de empleados.
- Prioridad: Alta
- Versión afectada: 0.2.0-beta
- Versión corregida: 0.2.1-beta

Conclusión.

La elaboración de un plan de pruebas integral y la definición de reglas claras para el versionado y manejo de incidencias constituyen actividades cruciales y distintivas en el desarrollo de software de alta calidad. En el ámbito laboral, estos elementos no solo aseguran la entrega de productos confiables, seguros y satisfactorios para el cliente, sino que también optimizan el uso de recursos, reducen el retrabajo, minimizan riesgos operativos y fortalecen la reputación profesional de equipos y organizaciones. La aplicación sistemática de estos principios permite transformar el caos potencial de un proyecto en un flujo estructurado, donde cada defecto es una oportunidad de mejora y cada liberación es un paso validado hacia la excelencia. En la vida cotidiana, más allá del contexto tecnológico, los conceptos de planificación detallada, control de cambios, documentación y mejora continua resultan igualmente valiosos para gestionar proyectos personales, académicos o comunitarios con mayor eficacia, previsión y menores imprevistos. Este proyecto final no solo sintetiza y aplica los conocimientos adquiridos a lo largo del curso de Aseguramiento de la Calidad, sino que también simula un escenario real de la industria, preparando al futuro profesional para enfrentar los desafíos de un entorno donde la calidad no es un lujo, sino un requisito fundamental para la competitividad y la sostenibilidad. La integración de la calidad desde las primeras etapas del ciclo de vida del software deja de ser una opción para convertirse en la esencia de un desarrollo responsable y exitoso.

Referencias.

- Pressman, R. (2005). *Software Engineering: A Practitioner's Approach* (6ª ed.). McGraw-Hill.
- IEEE (1998). *IEEE Standard 829-1998 for Software Test Documentation*. IEEE.
- ITIL Foundation (2011). *ITIL Service Operation*. The Stationery Office.
- Semantic Versioning (2021). *Semantic Versioning 2.0.0*. Recuperado de https://semver.org